



School of Digital, Technologies and Arts
Department of Computing

COMP60010: Enterprise Cloud and Distributed Web Applications

Assignment Specification

Weighted at 50% of the module mark

Submission Guidelines

All submissions are only electronic and should be submitted via Blackboard. If you have any questions/queries, please contact the tutor.

Submission deadline: Thursday 11 June 2026 (23:59)

Learning outcomes being assessed by this assignment are:

3. DESIGN, IMPLEMENT AND TEST A WEB APPLICATION BASED ON THE CLOUD AND CLOUD SERVICES
4. DEVELOP, IMPLEMENT AND, TEST A DISTRIBUTED WEB APPLICATION UTILISING AN APPLICATION PROGRAM INTERFACE (API).

This is an **individual** assessment. Please check that your report and/or script/tool is entirely written by yourself and any quoted sections are referenced.

Case Study

Scenario: AeroLink Airline Systems Platform

You are part of an advanced software architecture team at **AeroLink**, a global airline systems and aviation technology provider supporting airlines and airport operators across multiple regions.

AeroLink delivers critical digital services such as flight booking and ticketing, passenger check-in, baggage tracking, flight operations management, and integration with third-party systems including airports, immigration authorities, and payment providers.

AeroLink currently operates on a **monolithic architecture**, which is increasingly unable to support the organisation's requirements for:

- High availability
- Global scalability
- Fault tolerance
- Real-time data synchronisation

To address these challenges, AeroLink is transitioning to a **cloud-native, microservices-based architecture** that integrates cloud services and distributed applications through secure and scalable APIs.

Your task is to **design and implement a cloud-based distributed web application** that meets AeroLink's operational, scalability, and regulatory requirements.

Assignment Tasks

1. Cloud-Based Web Application Design

Design a comprehensive **cloud-based architecture** for AeroLink using appropriate cloud services. Your design should consider:

- Microservices architecture
- Containerisation (e.g., Docker, Kubernetes)
- Serverless computing (e.g., AWS Lambda)
- Cloud-managed databases (e.g., Amazon Aurora, DynamoDB)

The design must clearly address:

- High availability
- Multi-region deployment
- Horizontal scalability

Architectural diagrams and explanations are required.

2. Distributed Web Application and API Design

Design and implement a **distributed system** composed of loosely coupled microservices that communicate using:

- RESTful APIs and/or
- Event-driven architecture (e.g., Kafka, AWS EventBridge)

Your solution must include:

- An API Gateway for routing and authentication
- API documentation (Swagger / OpenAPI)
- Secure service-to-service communication

3. Data Security, Compliance, and Consistency

Design a security model to protect sensitive customer data by addressing:

- Encryption at rest and in transit
- Authentication and authorisation (OAuth 2.0, JWT, RBAC)
- Compliance with regulations such as **GDPR** and **PCI-DSS**

Discuss data consistency challenges in distributed systems and justify your chosen approach (e.g., eventual consistency, Saga pattern, CQRS).

4. Real-Time Data Synchronisation

Implement real-time data synchronisation mechanisms to ensure that:

- Flight seat availability updates
- Baggage status changes
- Flight schedule and pricing updates

are reflected across services with minimal delay. Use event-driven cloud services where appropriate.

5. Fault Tolerance and Resilience

Demonstrate how your system achieves fault tolerance through:

- Retry policies and circuit breakers
- Auto-scaling and load balancing

- High-availability zones and disaster recovery strategies

6. Performance and Scalability Testing

Conduct performance testing, including:

- Load testing
- Stress testing

Analyse latency, throughput, and error rates, and discuss potential performance improvements.

7. Monitoring and Observability

Implement monitoring and observability using cloud-native tools such as:

- AWS CloudWatch

Include logging, metrics, and distributed tracing to demonstrate system health and fault diagnosis.

8. Testing Strategy

- Unit testing of individual microservices
- Integration testing of APIs and cloud services
- API testing using tools such as Postman or Swagger

Provide evidence and discussion of test results.

Deliverables

1. **Final Report (PDF)**
 - Architecture design and justification
 - Security and compliance approach
 - Implementation overview
 - Testing and performance evaluation
 - Challenges and future improvements
2. **Source Code (ZIP)**
 - Microservices implementation

- API definitions and documentation
- Configuration and deployment files
- 3. **Presentation Slides**
 - 15-minute presentation summarising the solution

Assessment Criteria

Component	Weight
Architecture Design	20%
Implementation	40%
Testing and Results	20%
Presentation & Viva	20%